



Deep Learning – Learning on sets and graphs

Vlado Menkovski

Example task: Improved Red-shift Estimation Using Clustering Information



Develop a model that can estimate the amount of galaxy red-shift from photometric measurements.



Red-shift is an increase in the wavelength of light due to expansion of the Universe. The amount of red-shift of the light from distant galaxies is related to their age and distance.

Example task: Improved Red-shift Estimation Using Clustering Information



For each galaxy we have 17 photometric features



The galaxies are (partially) labelled with precise red-shift estimation based on spectroscoping measurement

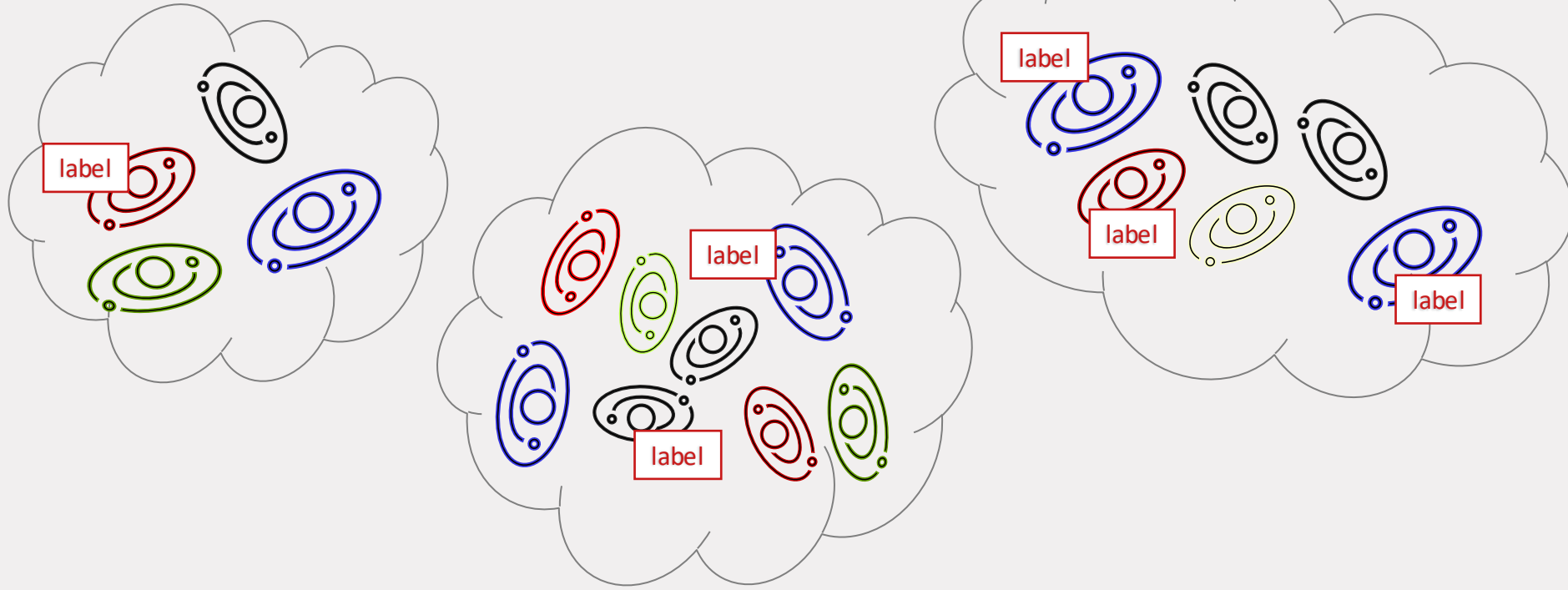


The galaxies are grouped in clusters of galaxies

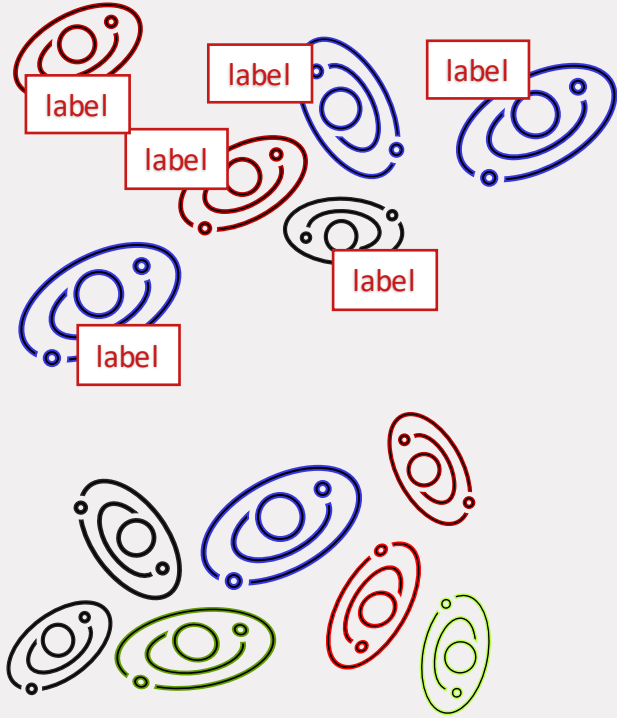


Consider the properties of the other galaxies in the cluster to improve the model's prediction

Galaxy red-shift dataset

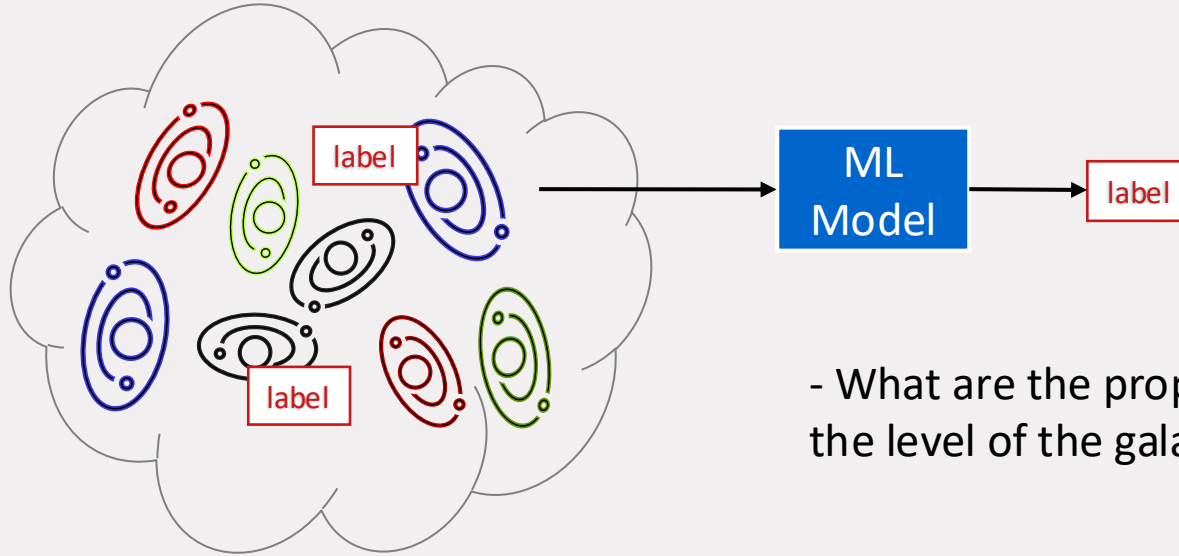


Galaxy red-shift model (ignoring the clusters)



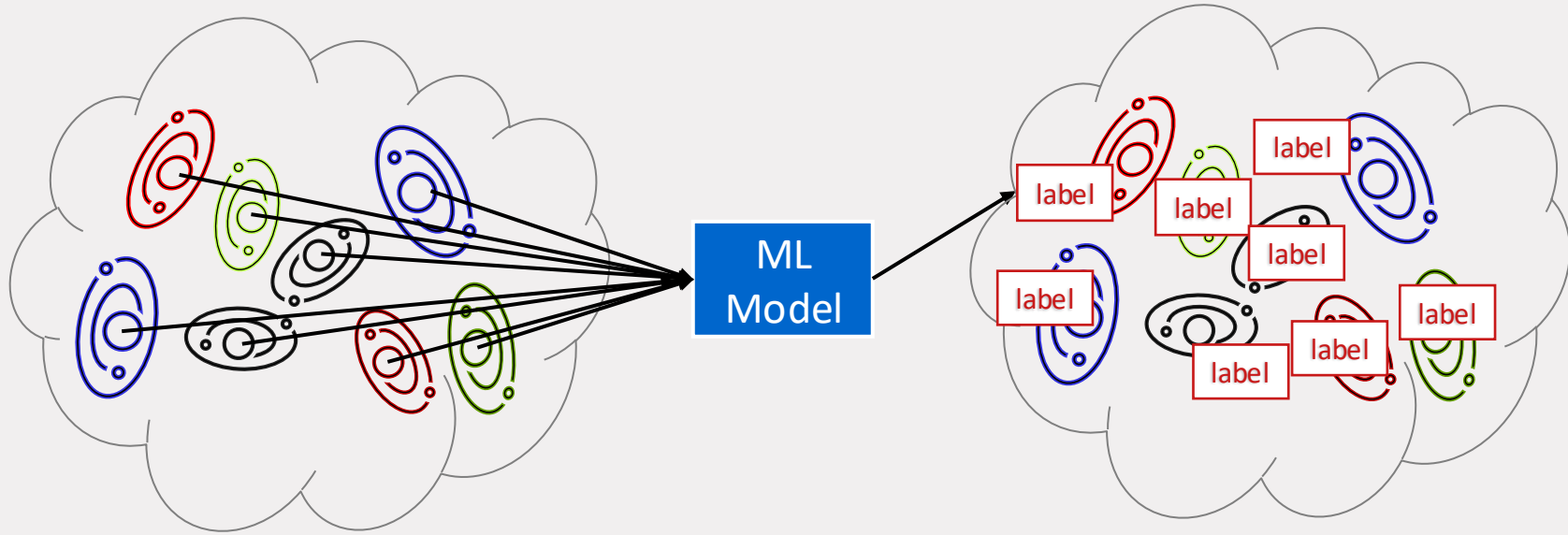
- What are the advantages and disadvantages to using only the labelled subset of the data?

Galaxy red-shift model



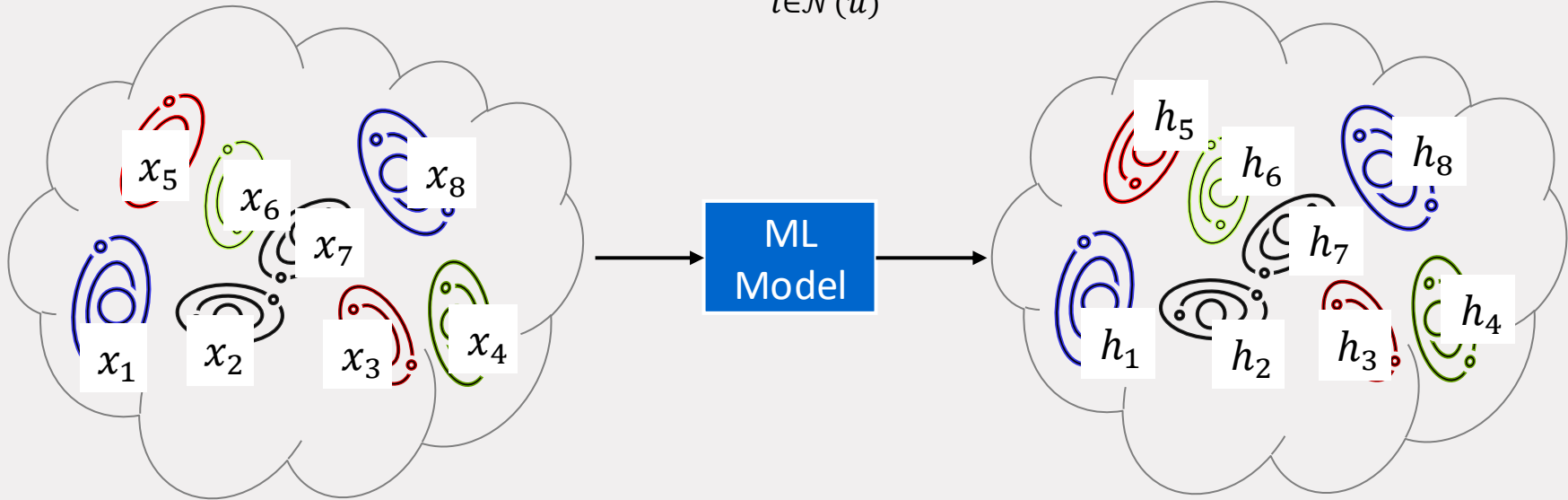
- What are the properties of the data domain at the level of the galaxy clusters?

Galaxy red-shift model



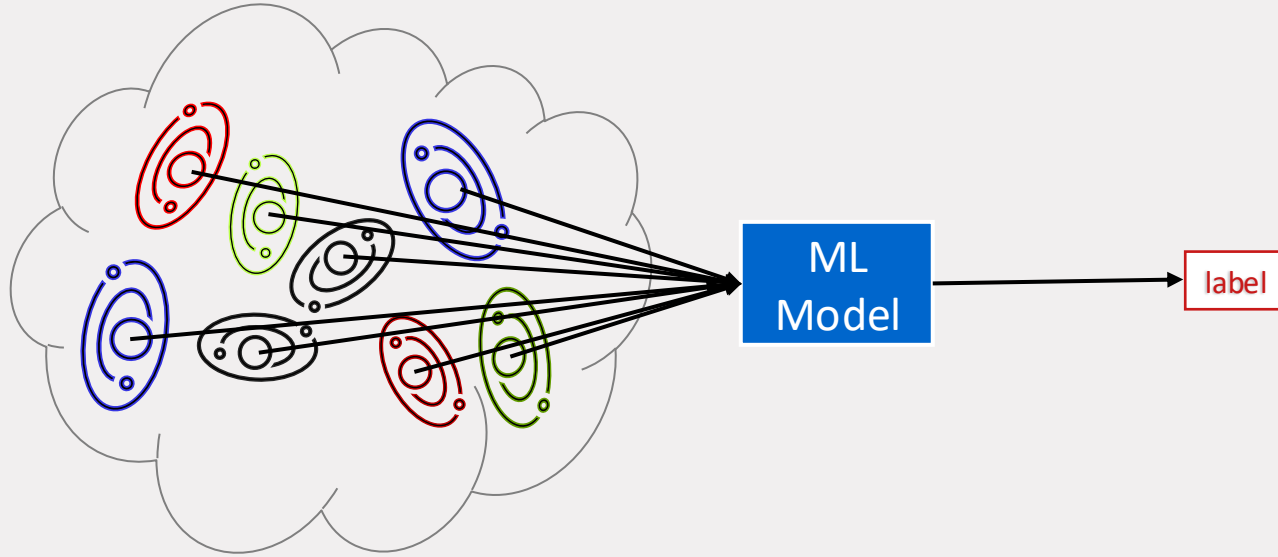
Deep Set model

$$h_u = \phi(x_u, \bigcup_{i \in \mathcal{N}(u)} \psi(x_i))$$

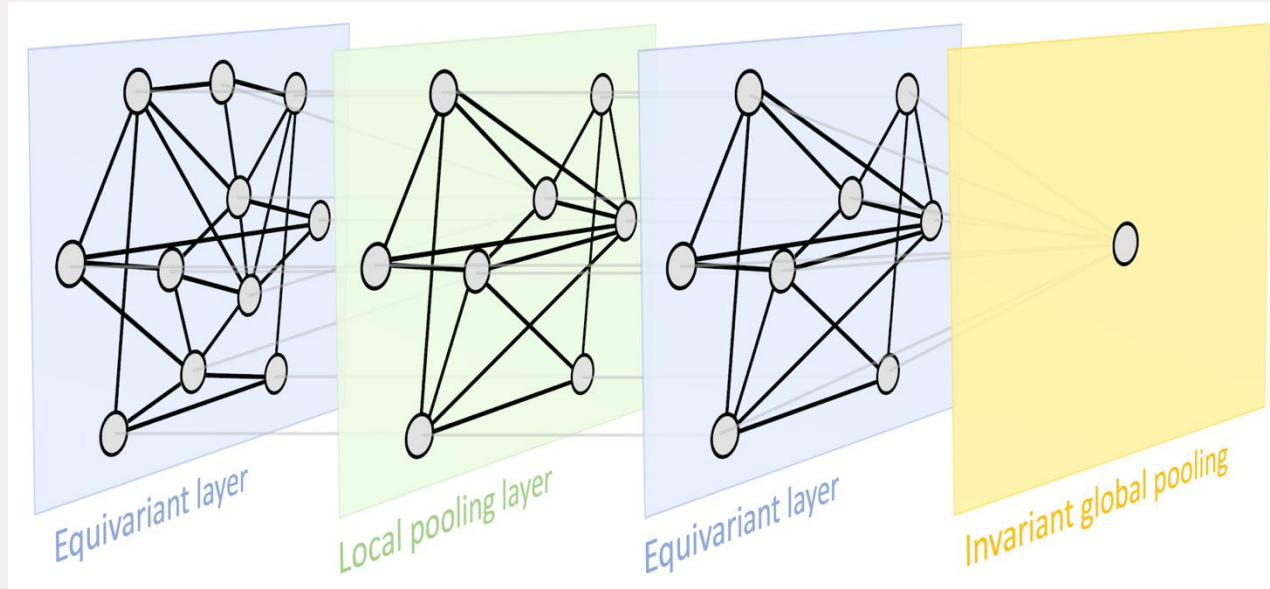


Zaheer, Manzil, et al. "Deep sets." *Advances in neural information processing systems* 30 (2017).

Galaxy red-shift dataset



Geometric deep learning framework



Equivariant layers for representation learning. Invariant layers for global model invariance.

Example task: Publication field detection

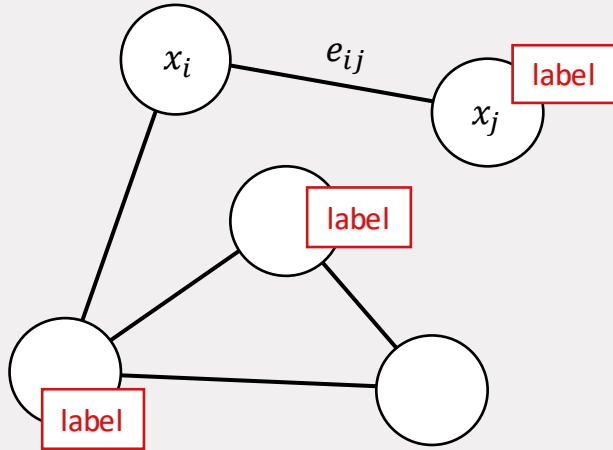


Develop a model that classifies the publications in one of seven different categories.



The Cora dataset consists of scientific publications and a citation network between them. Each publication in the dataset is described by a 0/1-valued bag of words representation.

Cora dataset

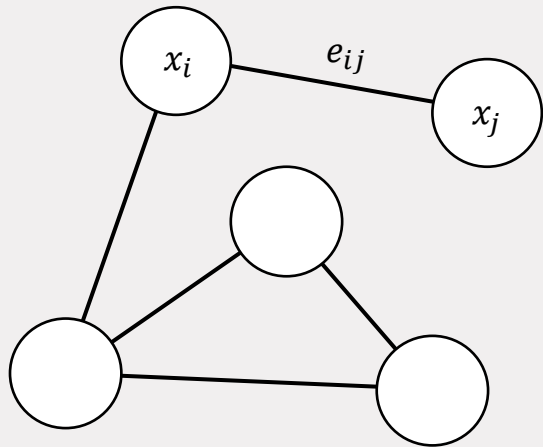


- The Cora dataset consists of 2708 scientific publications classified into one of seven classes.
- The citation network consists of 5429 links.
- Each publication in the dataset is described by a 0/1-valued word vector indicating the absence /presence of the corresponding word from the dictionary. The dictionary consists of 1433 unique words.

7 Classes:

Case_Based, Genetic_Algorithms, Neural_Networks, Probabilistic_Methods, Reinforcement_Learning, Rule_Learning, Theory

What is a graph?



$$\mathcal{G} = (\mathcal{V}, \mathcal{E})$$

$\mathcal{V}$ Vertecies (nodes), Objects

$\mathcal{E} = \{(i, j): i, j \in \mathcal{V}\}$ Edges,
relationships between nodes

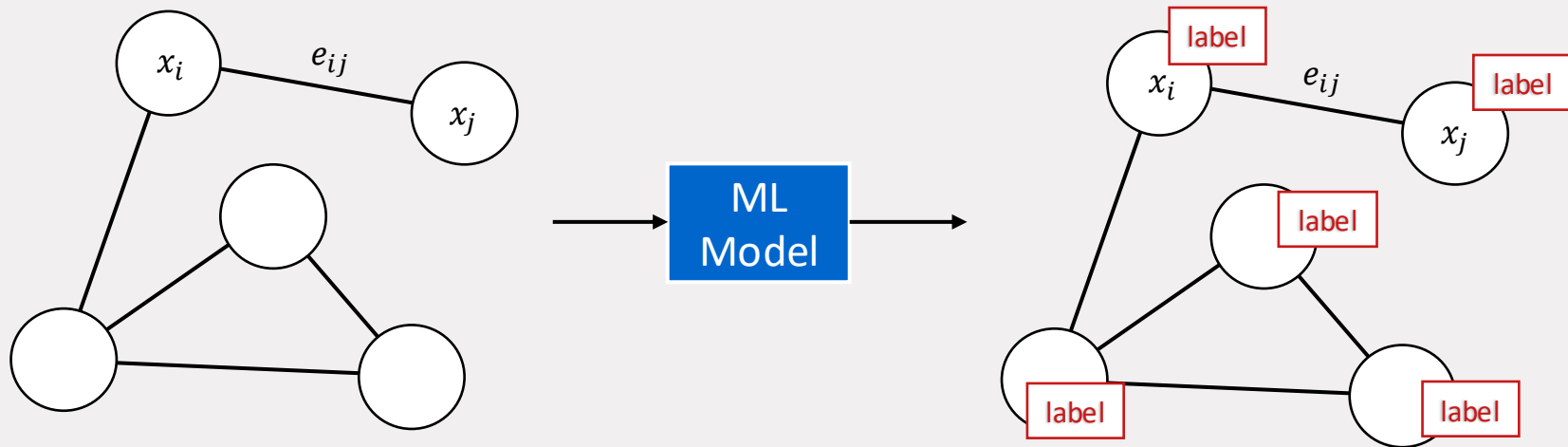
$\mathcal{N}(i) = \{j: (i, j) \in \mathcal{E}\}$ Set of
nodes neighbouring node i

$x_i \in R^d$ Node features

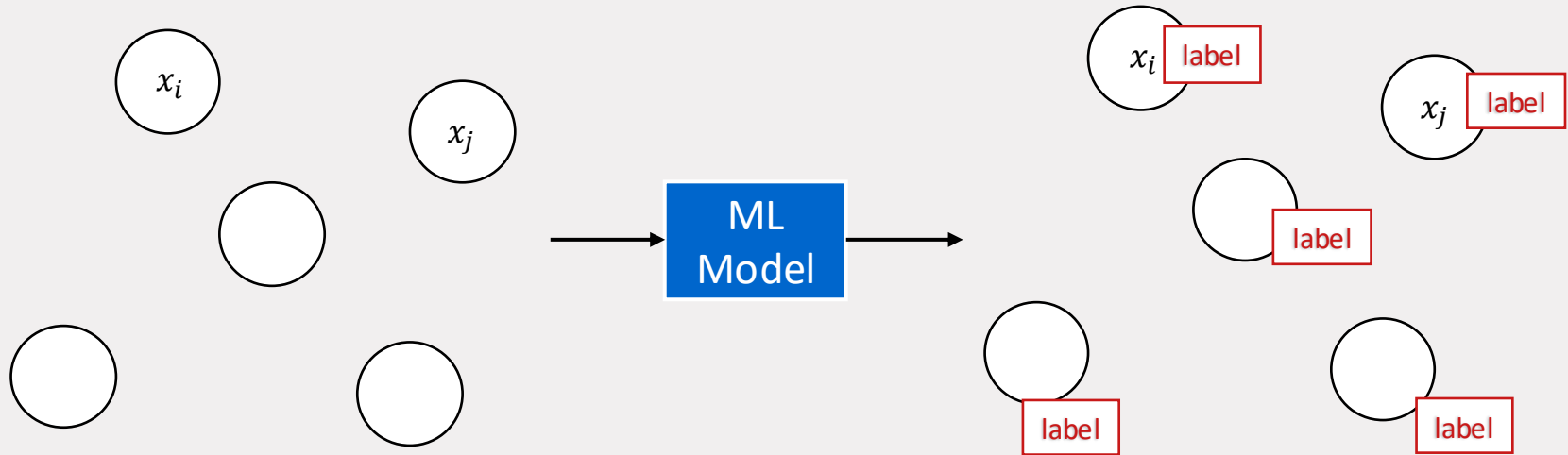
e_{ij} Edge or edge features

$e_{ij} \in R^{d'}$ where $d' = 1$ for a weighted graph

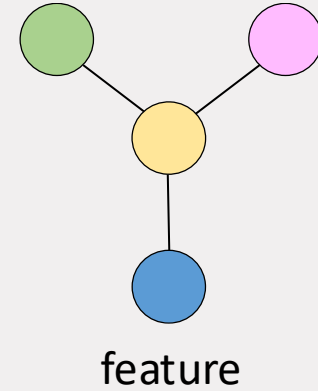
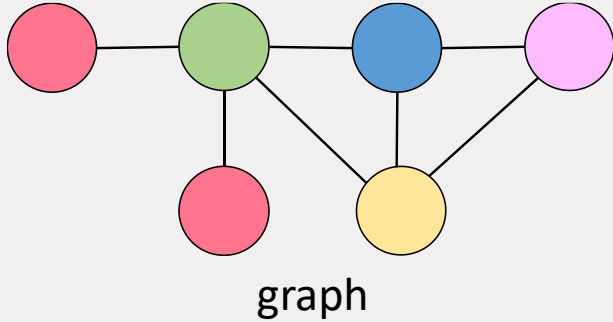
Cora model



Cora model (ignore graph structure)



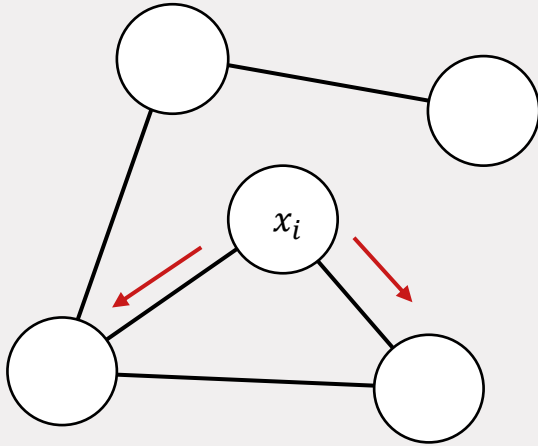
The graph domain



- Permutational invariance
- Variable Neighborhood size

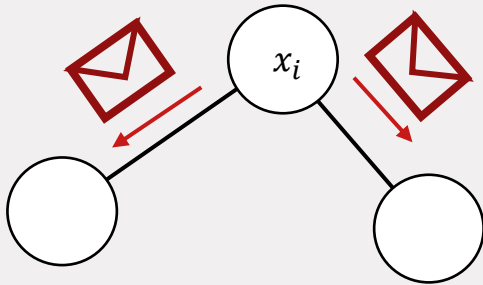
Neural Message Passing

Emit a message to all neighbours



Neural Message Passing

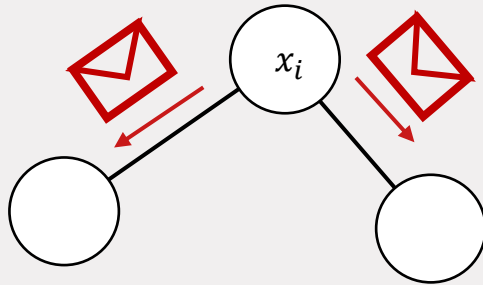
Emit a message to all neighbours



Neural Message Passing

Emit a message to all neighbours

Node transfer function processes each message



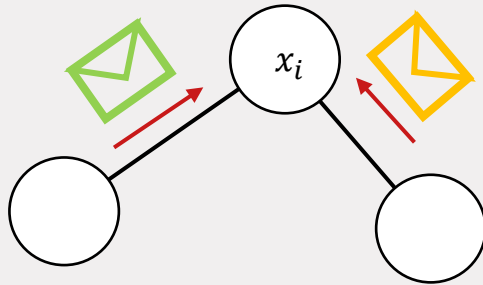
$$\phi(\text{envelope}) \rightarrow \text{envelope}$$

Neural Message Passing

Emit a message to all neighbours

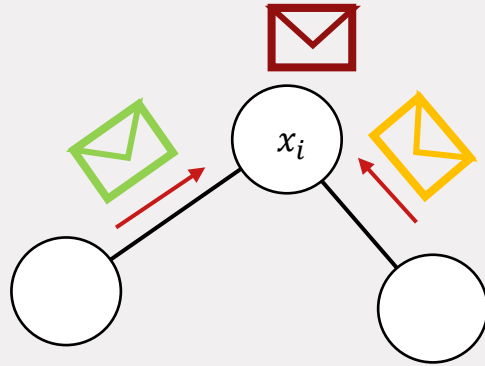
Node transfer function processes each message

Node aggregation function produces a representation of all neighbourhood messages



$$U \left(\text{green envelope}, \text{yellow envelope} \right) \longrightarrow \text{blue envelope}$$

Neural Message Passing

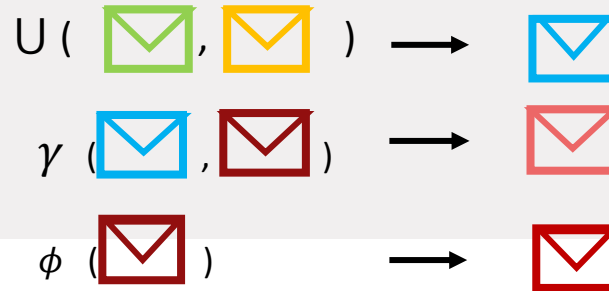


Emit a message to all neighbours

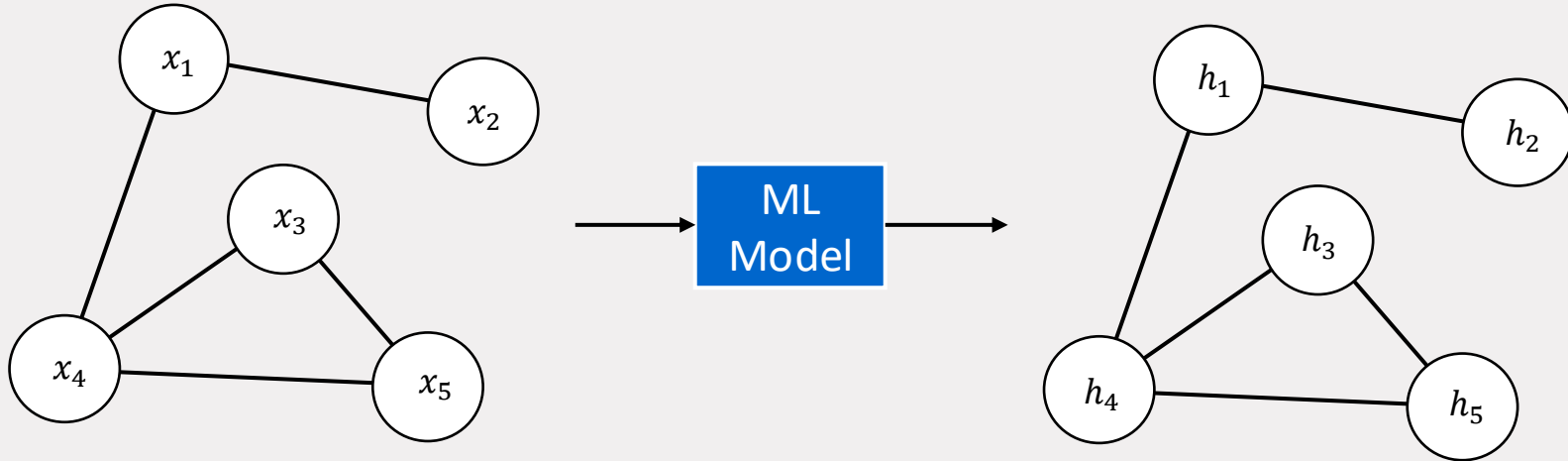
Node transfer function processes each message

Node aggregation function produces a representation of all neighbourhood messages

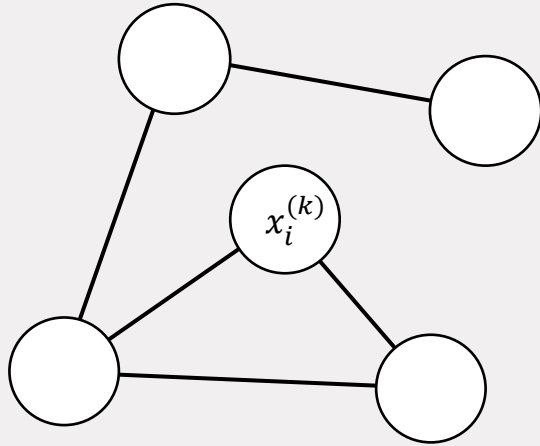
Node update function produces a new node representation



Equivariant graph layer



Neural Message Passing



Emit a message to all neighbours

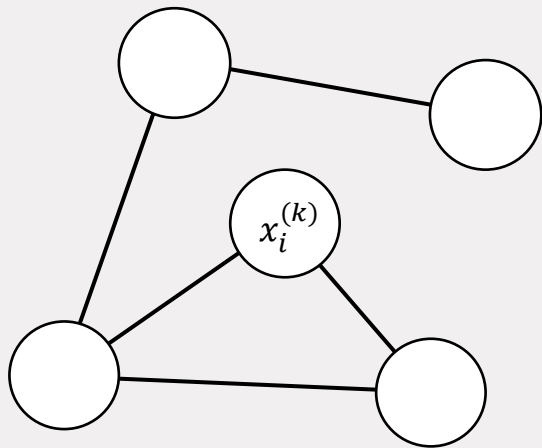
Node transfer function processes each message

Node aggregation function produces a representation of all neighbourhood messages

Node update function produces a new node representation

Repeat the emission k-iterations

Neural Message Passing



$$x_i^{(k)} = \gamma^{(k)}(x_i^{(k-1)}, \bigcup_{j \in \mathcal{N}(i)} \phi^{(k)}(x_i^{(k-1)}, x_j^{(k-1)}, e_{j,i}))$$

$\phi^{(k)}$ - Edge transfer function at iteration k

$\gamma^{(k)}$ - Node update function at iteration k

$\bigcup$ - Message aggregation function

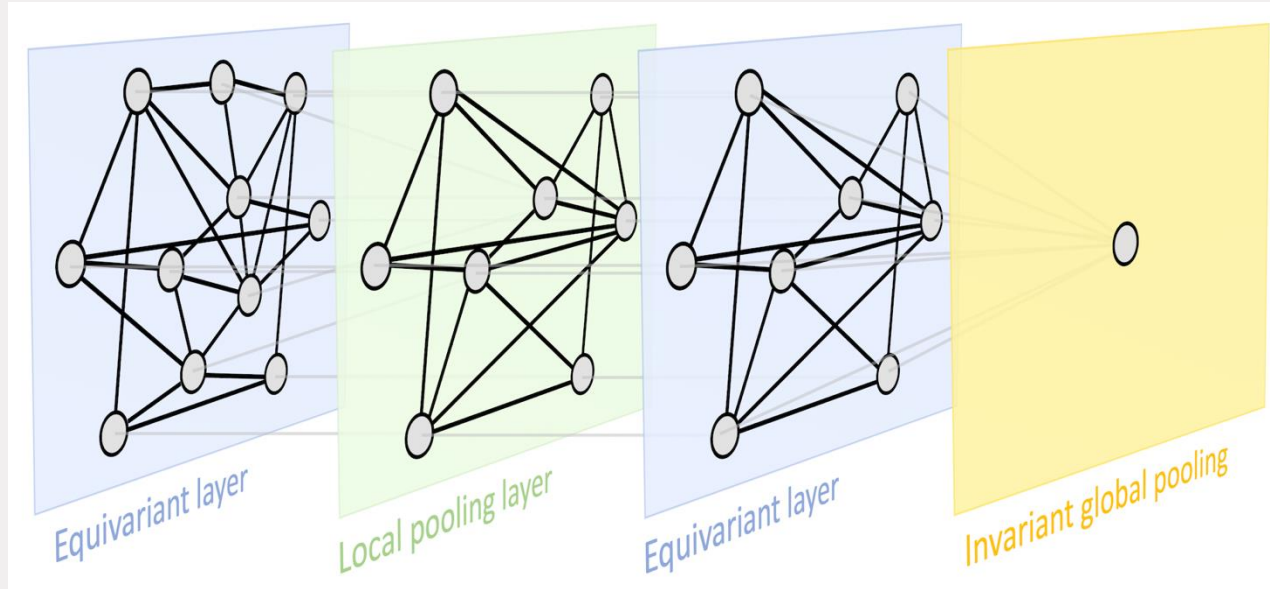
$\mathcal{N}(i)$ - Neighbour nodes to node i

$x_i^{(k)}$ - Representation of node i at iteration k

$e_{j,i}$ - Representation of edge between nodes i and j

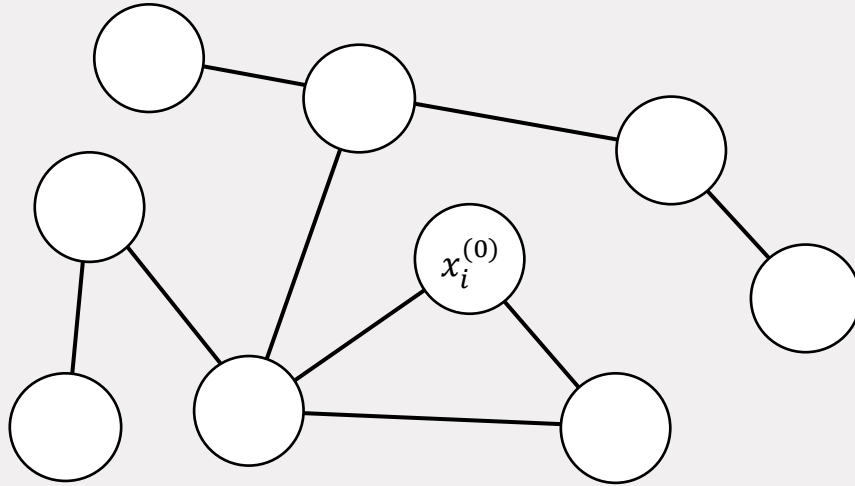
Gilmer, Justin, et al. "Neural message passing for quantum chemistry." *International conference on machine learning*. PMLR, 2017.

Geometric deep learning framework

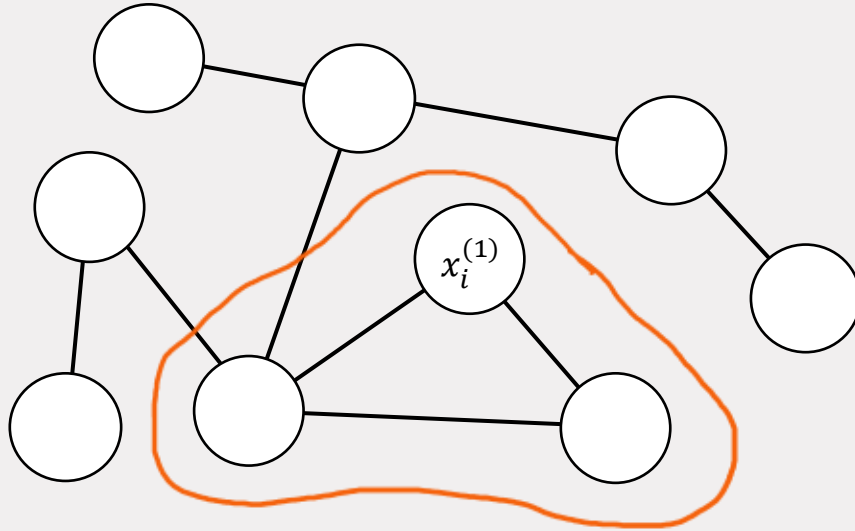


Equivariant layers for representation learning. Invariant layers for global model invariance.

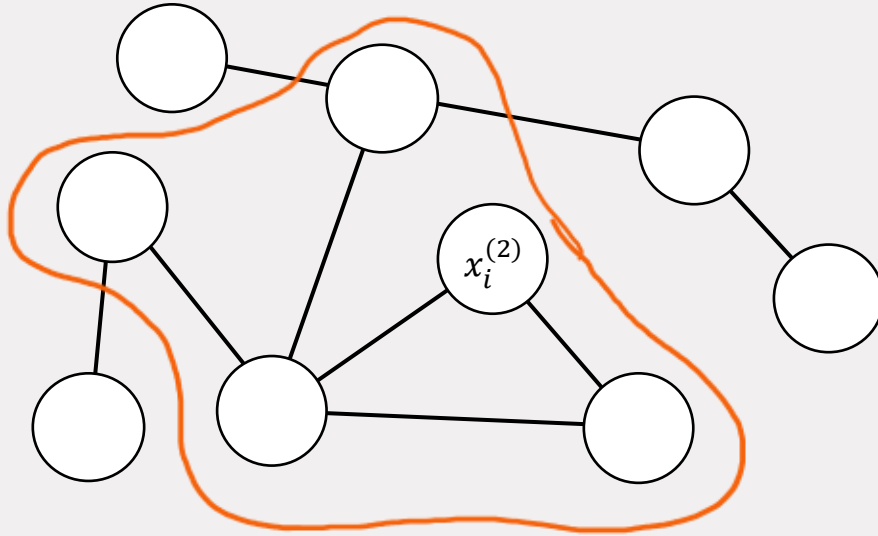
Neural Message Passing



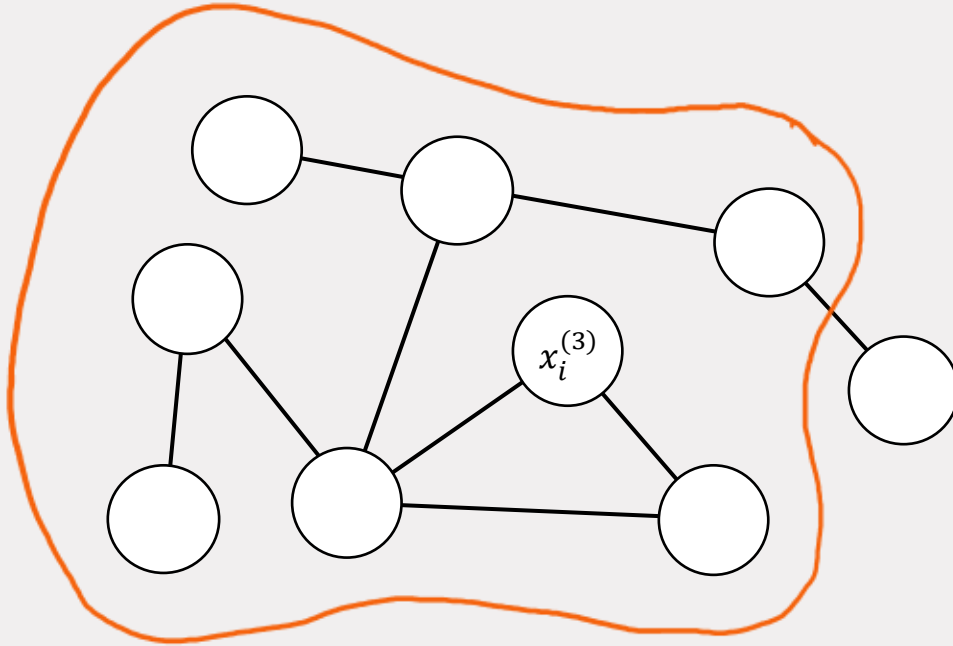
Neural Message Passing



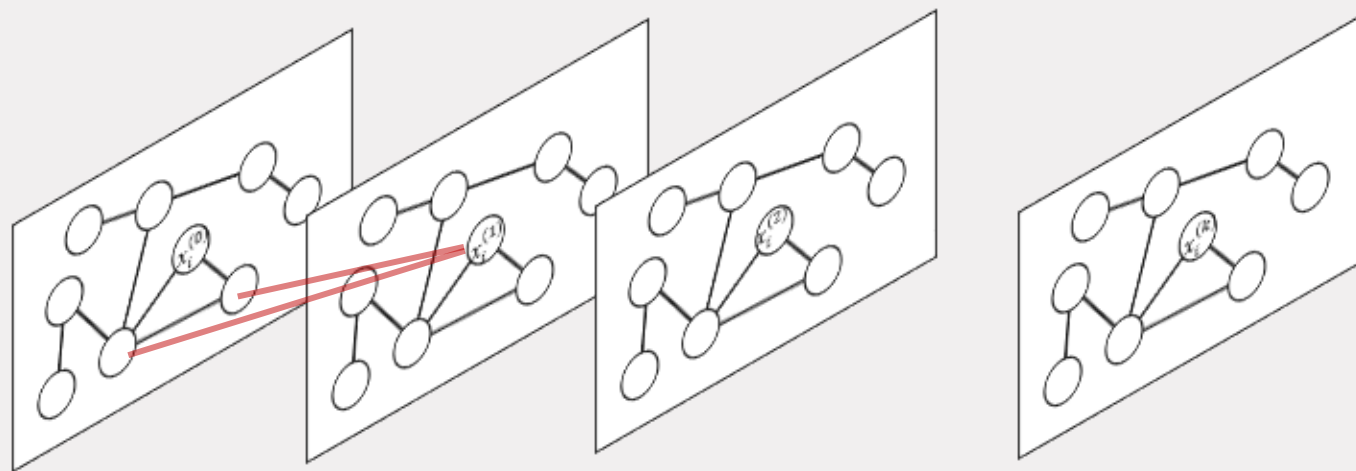
Neural Message Passing



Neural Message Passing

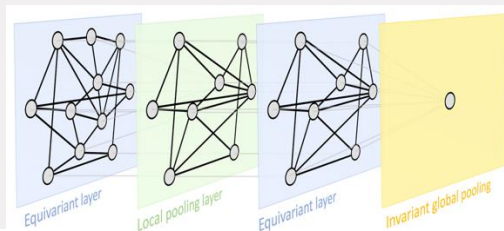
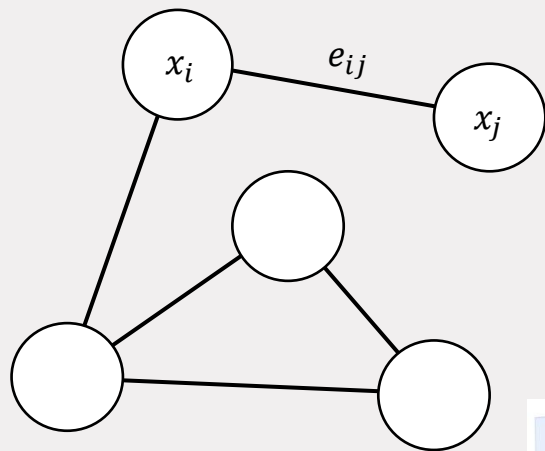


Neural Message Passing



Operating globally on a graph

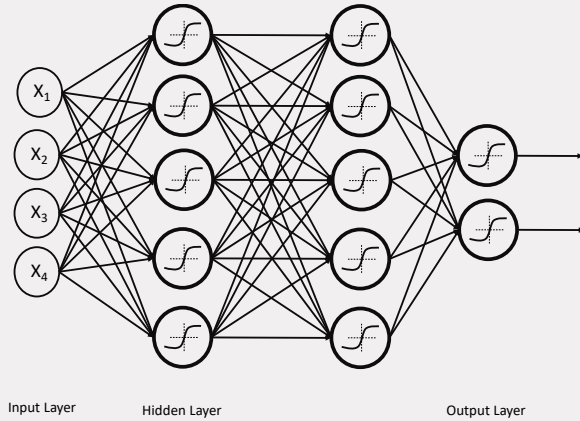
$$x_i^{(k)} = \gamma^{(k)} \left(x_i^{(k-1)}, \bigcup_{j \in \mathcal{N}(i)} \phi^{(k)}(x_i^{(k-1)}, x_j^{(k-1)}, e_{j,i}) \right)$$



$$x_i^{(k)} = \xi_{i \in \mathcal{V}}(x_i^{(k)})$$

read-out function

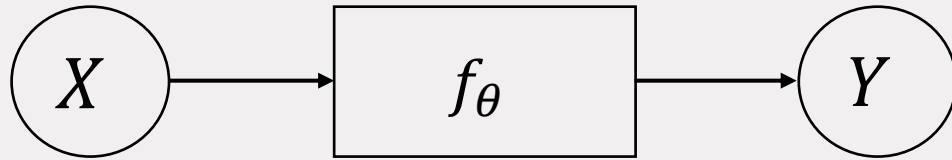
Hard weights



$$Y = f_{\theta}(X)$$

θ weights are computed during training

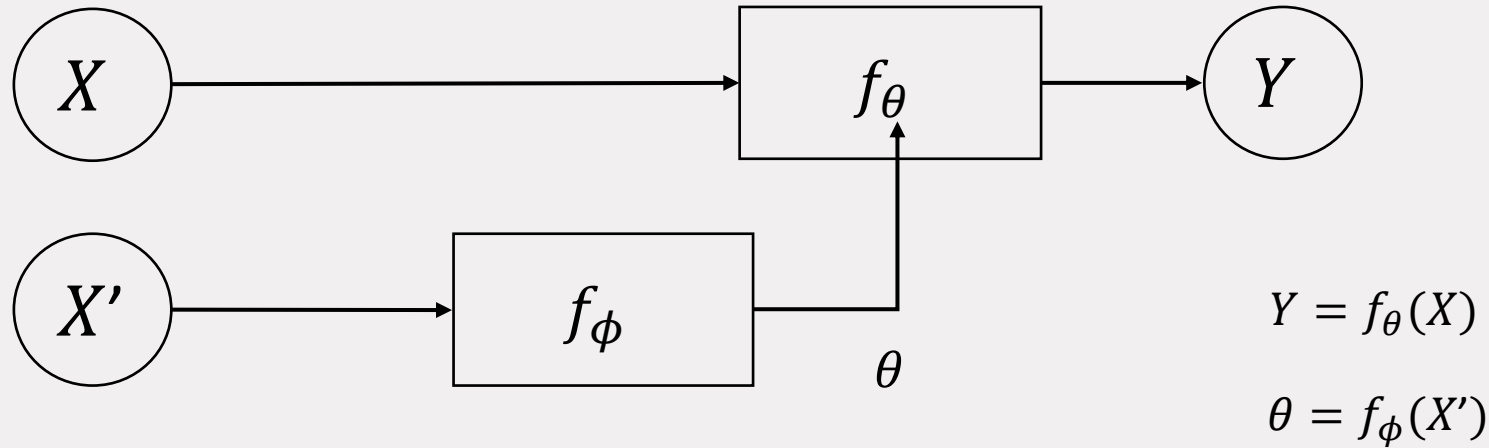
Hard weights



$$Y = f_\theta(X)$$

θ weights are computed during training

Soft weights

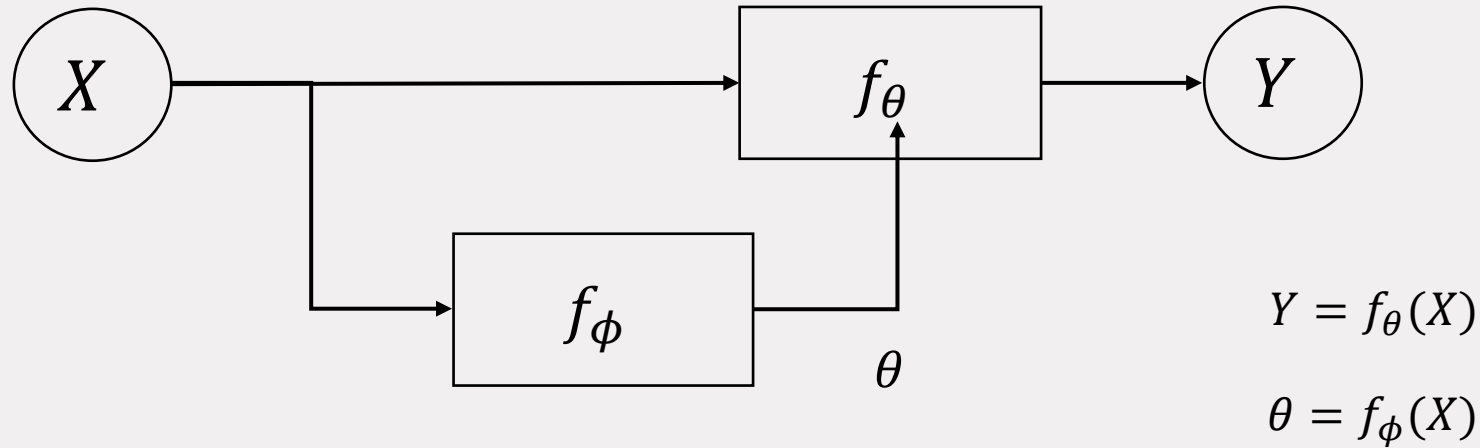


ϕ weights are computed during training

θ are computed during inference

The attention mechanisms is a specific architecture that uses these ‘soft weights’

Soft weights

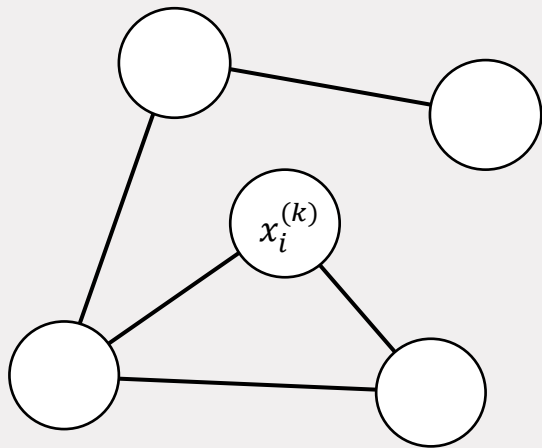


ϕ weights are computed during training

θ are computed during inference

The attention mechanisms is a specific architecture that uses these ‘soft weights’

Neural Message Passing



$$x_i^{(k)} = \gamma^{(k)}(x_i^{(k-1)}, \bigcup_{j \in \mathcal{N}(i)} \phi^{(k)}(x_i^{(k-1)}, x_j^{(k-1)}, e_{j,i}))$$

$\phi^{(k)}$ - Edge transfer function at iteration k

$\gamma^{(k)}$ - Node update function at iteration k

$\bigcup$ - Message aggregation function

$\mathcal{N}(i)$ - Neighbour nodes to node i

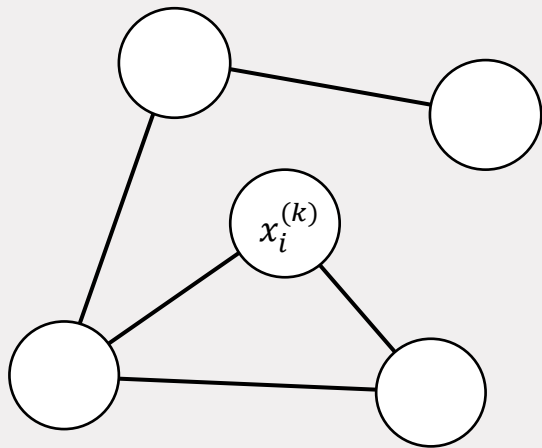
$x_i^{(k)}$ - Representation of node i at iteration k

$e_{j,i}$ - Representation of edge between nodes i and j

Gilmer, Justin, et al. "Neural message passing for quantum chemistry." *International conference on machine learning*. PMLR, 2017.

Neural Message Passing

$$x_i^{(k)} = \gamma^{(k)} \left(x_i^{(k-1)}, \bigcup_{j \in \mathcal{N}(i)} \phi^{(k)}(x_i^{(k-1)}, x_j^{(k-1)}, e_{j,i}) \right)$$



All neighbours are treated equally

Can a more expressive neighbourhood representation?

Graph Attention Networks

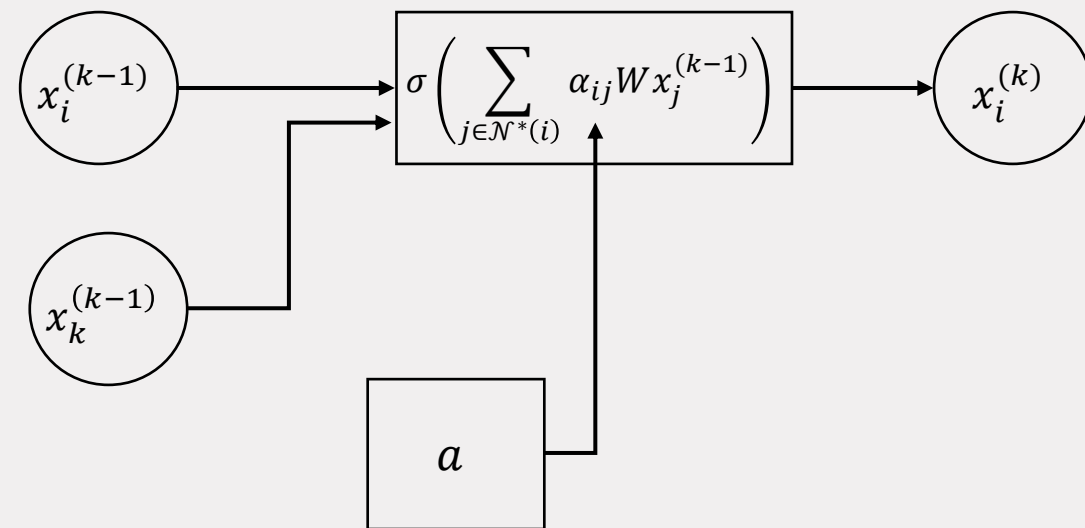
$$x_i^{(k)} = \gamma^{(k)} \left(\bigcup_{j \in \mathcal{N}^*(i)} \alpha_{ij} \phi^{(k)}(x_i^{(k-1)}, x_j^{(k-1)}) \right)$$

different neighbours get
a different weight

$\mathcal{N}^*(i)$ - includes i

Veličković, Petar, et al. "Graph attention networks." arXiv preprint arXiv:1710.10903 (2017).

Graph Attention Networks

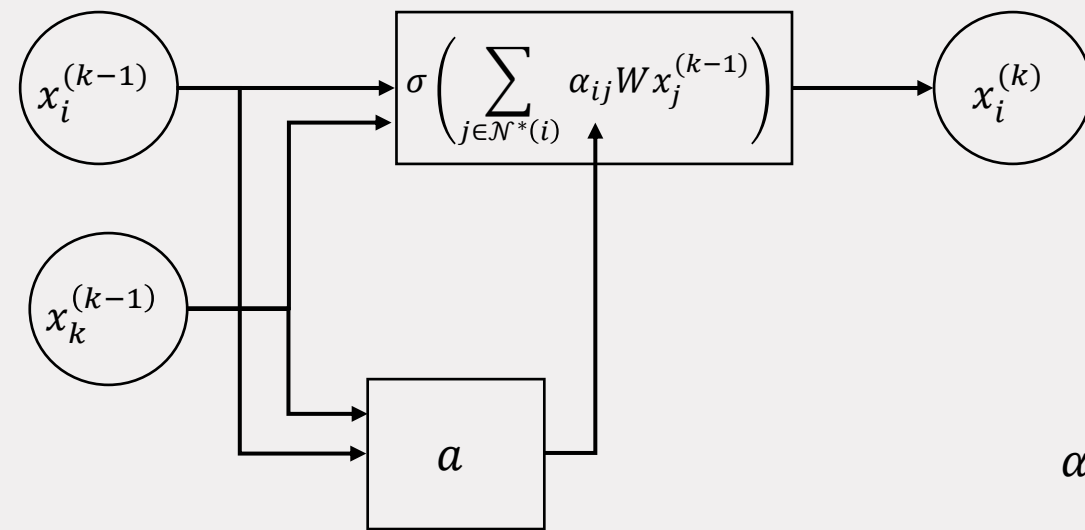


$$x_i^{(k)} = \sigma\left(\sum_{j \in \mathcal{N}^*(i)} \alpha_{ij} W x_j^{(k-1)}\right)$$

soft weights
attend to each neighbour

Veličković, Petar, et al. "Graph attention networks." arXiv preprint arXiv:1710.10903 (2017).

Graph Attention Networks



$$x_i^{(k)} = \sigma \left(\sum_{j \in \mathcal{N}^*(i)} \alpha_{ij} W x_j^{(k-1)} \right)$$

soft weights

attend to each neighbour

$$m_{ij} = a(W x_i^{(k-1)} || W x_j^{(k-1)})$$

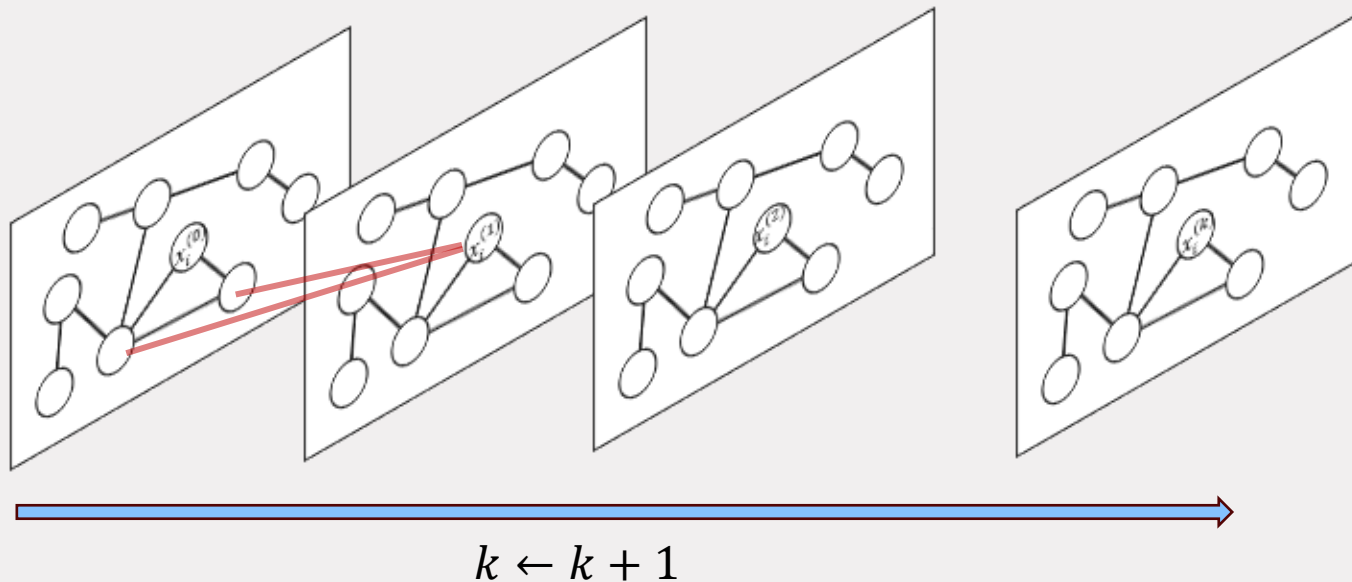
$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(m_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(m_{ik})}$$

Veličković, Petar, et al. "Graph attention networks." arXiv preprint arXiv:1710.10903 (2017).

GNN: Node embeddings computed in parallel

$$x_i^{(k)} = \gamma^{(k)} \left(x_i^{(k-1)}, \bigcup_{j \in \mathcal{N}(i)} \phi^{(k)}(x_i^{(k-1)}, x_j^{(k-1)}, e_{j,i}) \right)$$

Update rule can be applied to all nodes in the graph in parallel



GNN: Node embeddings computed in parallel

$$x_i^{(k)} = \gamma^{(k)} \left(x_i^{(k-1)}, \bigcup_{j \in \mathcal{N}(i)} \phi^{(k)}(x_i^{(k-1)}, x_j^{(k-1)}, e_{j,i}) \right)$$

$\bar{x} \in \mathbb{R}^{d \times n}$ - embeddings of all nodes

$A \in \mathbb{R}^{d \times n \times n}$ - all messages in one message passing step

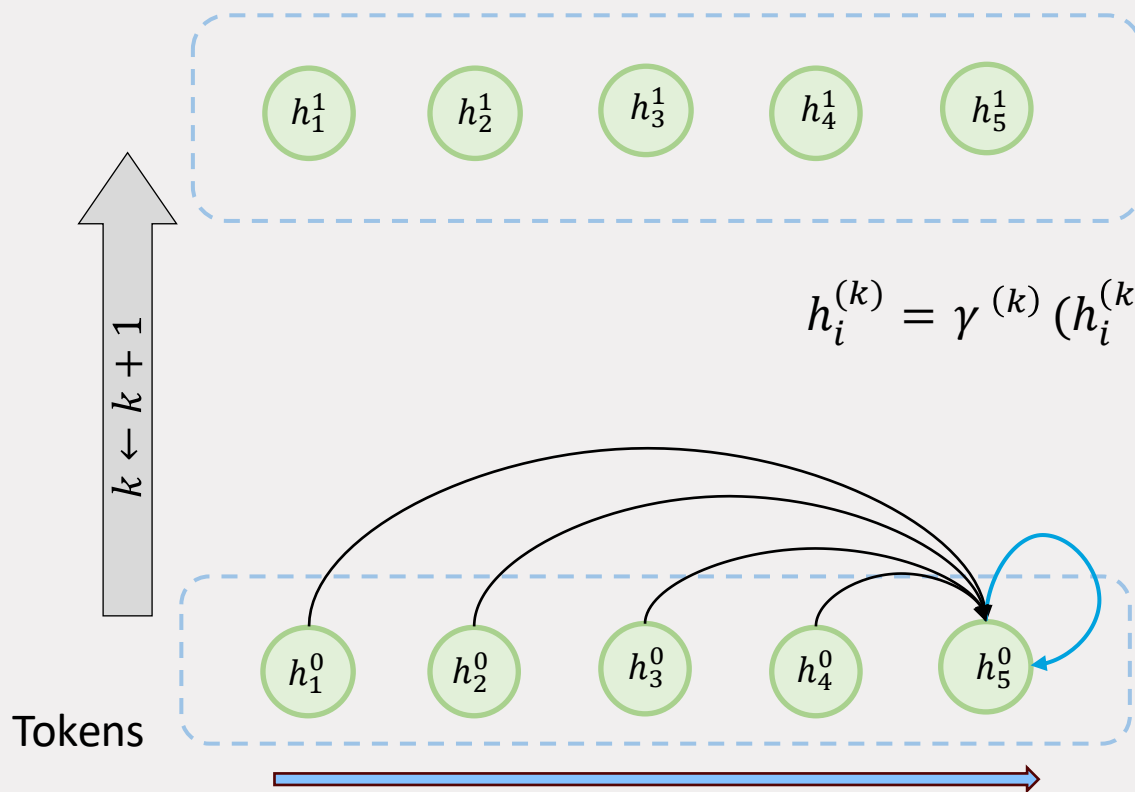
$\bigcup_j A \in \mathbb{R}^{d \times n}$ - neighbour embedding of all nodes in the graph


$$A = \phi^{(k)}(\bar{x}^{(k-1)}, \bar{e})$$


$$\bar{x}^{(k)} = \gamma^{(k)}(\bar{x}^{(k-1)}, \bigcup_j A)$$

Computed on parallel using matrix multiplication hardware

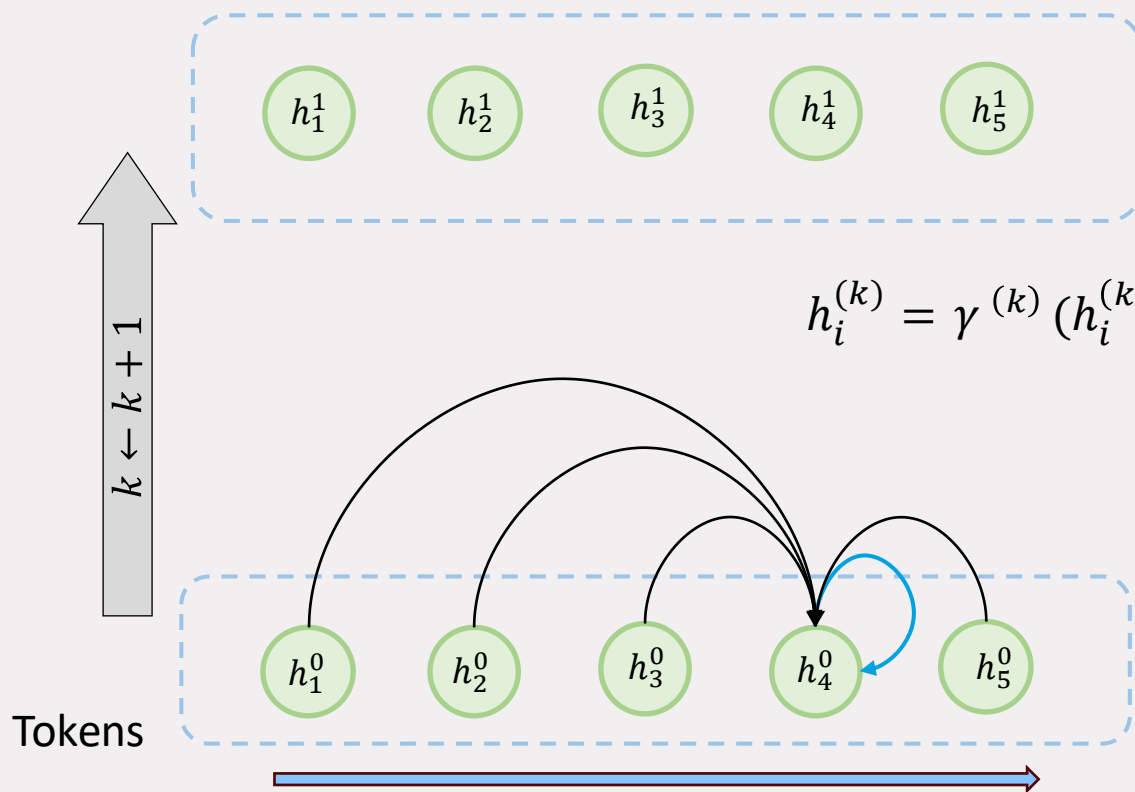
Process a sequence (as a set) – to parallelize the computation



$$h_i^{(k)} = \gamma^{(k)}(h_i^{(k-1)}, \bigcup_{j \in \mathcal{N}(i)} \phi^{(k)}(h_i^{(k-1)}, h_j^{(k-1)}, e_{j,i}))$$

$$h_i^{(k)} = \psi\left(h_i^{(k)}, \sum_{j \in \mathcal{N}(i)} \frac{1}{n} W h_j^{(k)}\right)$$

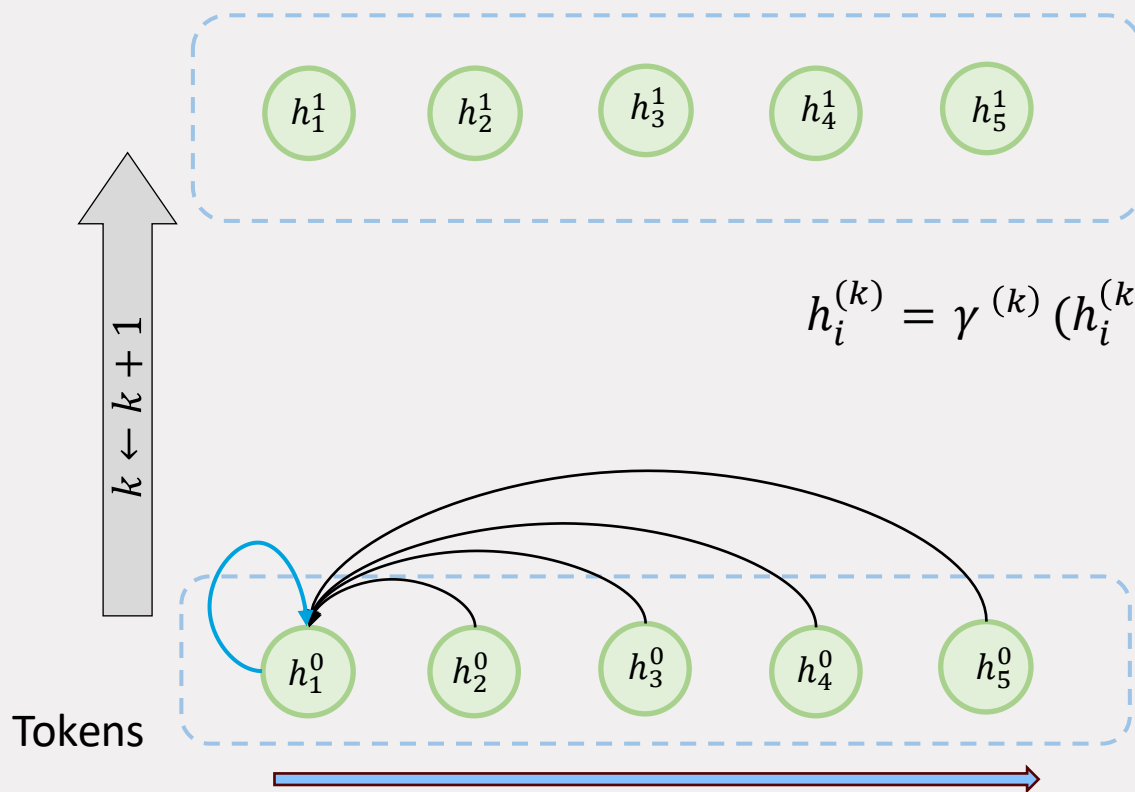
Process a sequence (as a set) – to parallelize the computation



$$h_i^{(k)} = \gamma^{(k)}(h_i^{(k-1)}, \bigcup_{j \in \mathcal{N}(i)} \phi^{(k)}(h_i^{(k-1)}, h_j^{(k-1)}, e_{j,i}))$$

$$h_i^{(k)} = \psi\left(h_i^{(k)}, \sum_{j \in \mathcal{N}(i)} \frac{1}{n} W h_j^{(k)}\right)$$

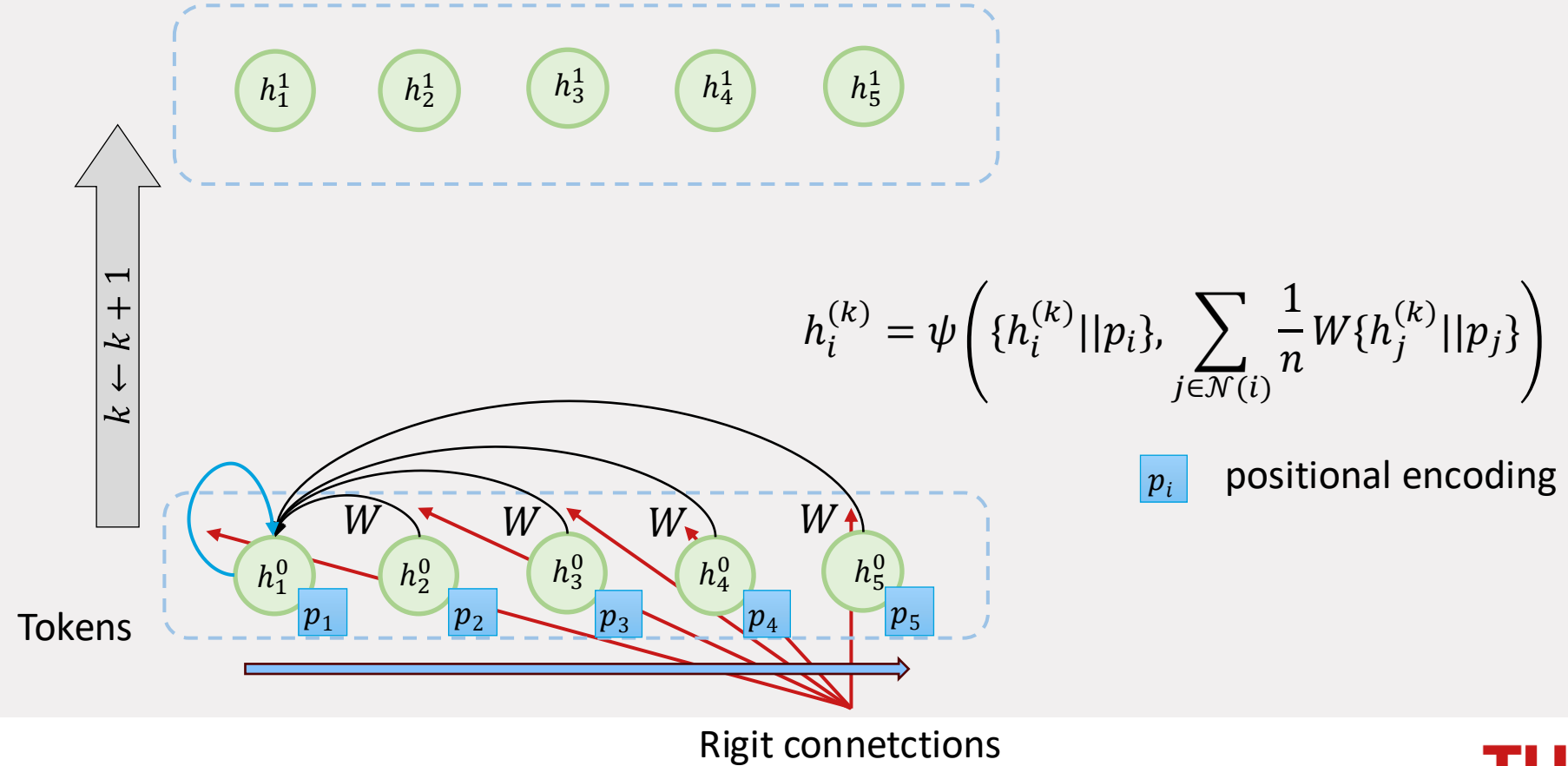
Process a sequence (as a set) – to parallelize the computation



$$h_i^{(k)} = \gamma^{(k)}(h_i^{(k-1)}, \bigcup_{j \in \mathcal{N}(i)} \phi^{(k)}(h_i^{(k-1)}, h_j^{(k-1)}, e_{j,i}))$$

$$h_i^{(k)} = \psi\left(h_i^{(k)}, \sum_{j \in \mathcal{N}(i)} \frac{1}{n} W h_j^{(k)}\right)$$

Process a sequence (as a set + positional encoding)

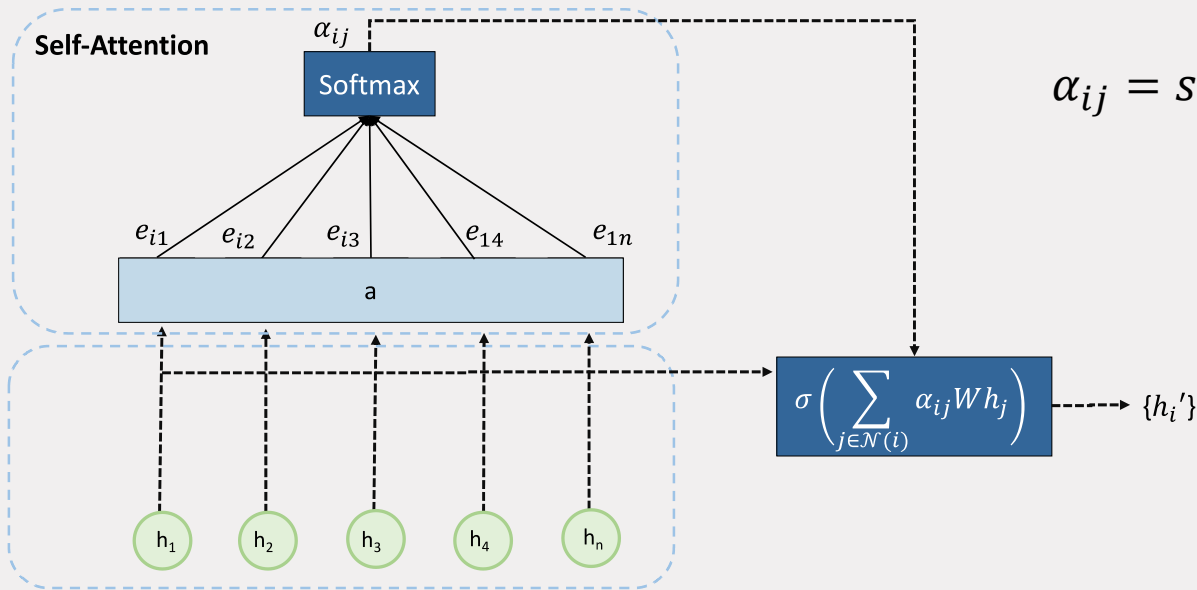


$$h_i^{(k)} = \psi \left(\{h_i^{(k)} || p_i\}, \sum_{j \in \mathcal{N}(i)} \frac{1}{n} W \{h_j^{(k)} || p_j\} \right)$$

p_i positional encoding

Self-attention

$$e_{ij} = a(W h_i || W h_j)$$

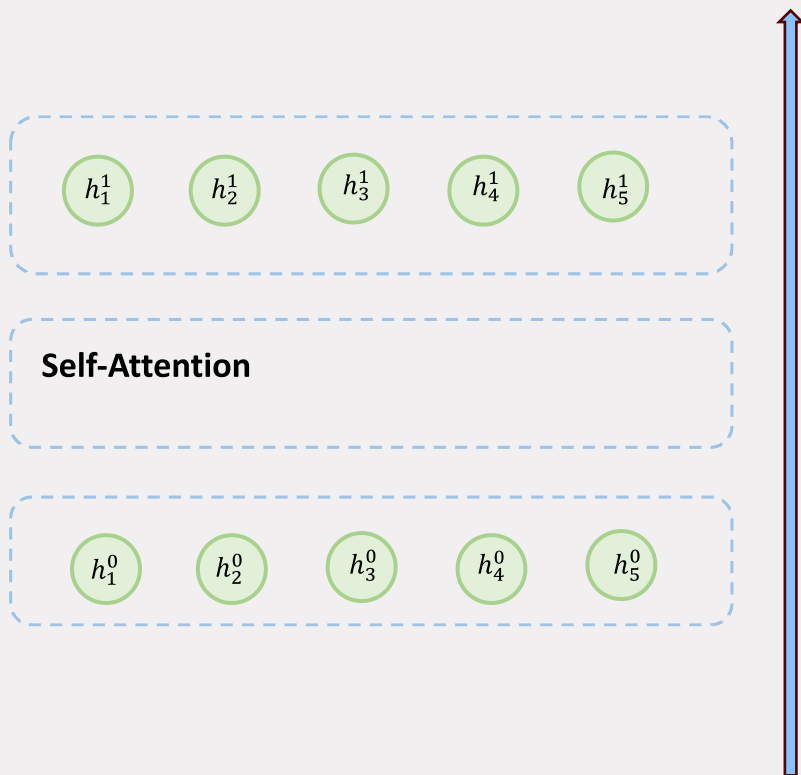


$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}^*(i)} \exp(e_{ik})}$$

$$h'_i = \sigma\left(\sum_{j \in \mathcal{N}^*(i)} \alpha_{ij} W h_j\right)$$

soft weights

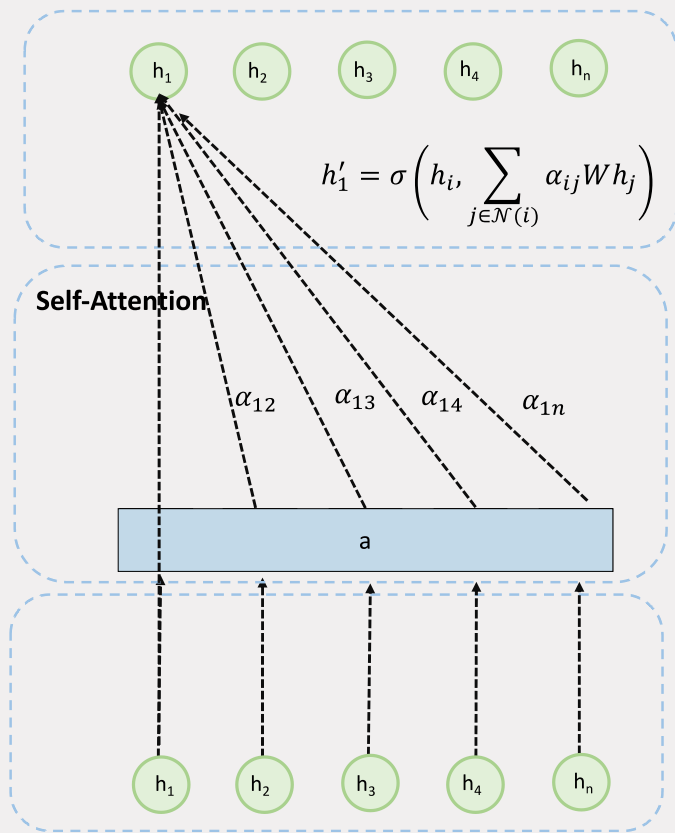
Self-attention



The self-attention layer updates the token value with the rest of the values of the sequence (set).

$$h'_i = \sigma \left(\sum_{j \in \mathcal{N}^*(i)} \alpha_{ij} W h_j \right)$$

Self-attention

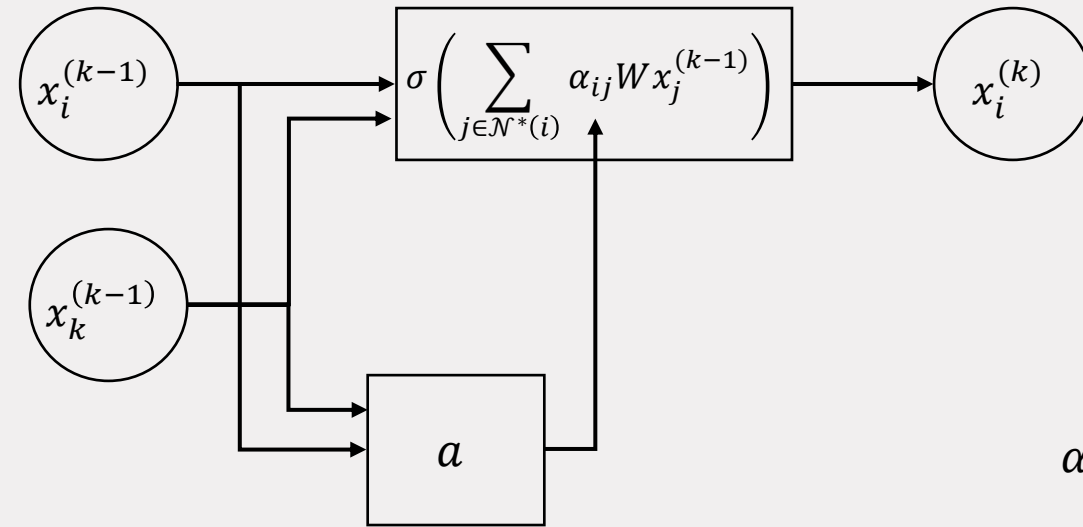


$$e_{ij} = a(W h_i || W h_j)$$

$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}^*(i)} \exp(e_{ik})}$$

$$h'_i = \sigma \left(\sum_{j \in \mathcal{N}^*(i)} \alpha_{ij} W h_j \right)$$

Graph Attention Networks and Self-attention equivalence



$$x_i^{(k)} = \sigma\left(\sum_{j \in \mathcal{N}^*(i)} \alpha_{ij} W x_j^{(k-1)}\right)$$

soft weights

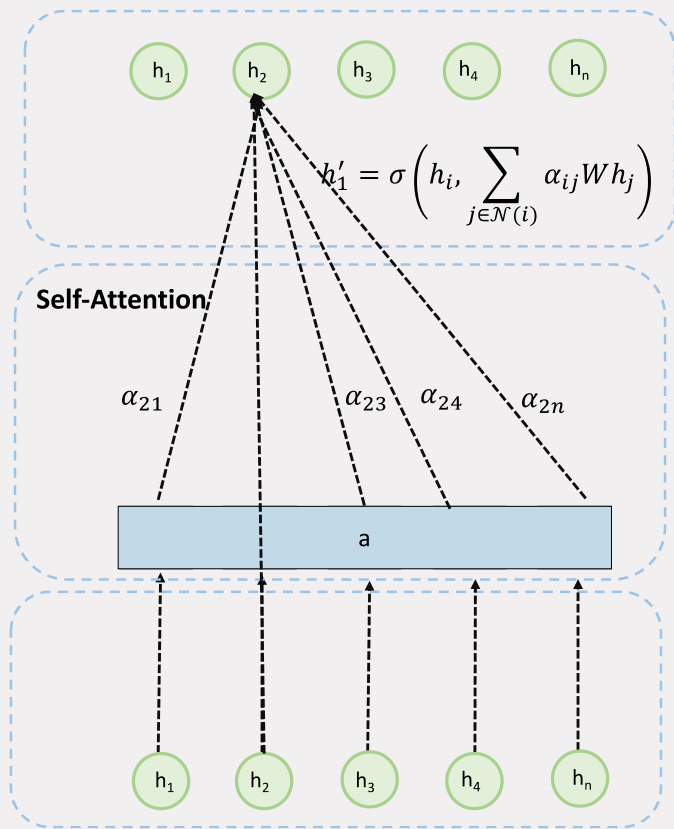
attend to each neighbour

$$m_{ij} = a(W x_i^{(k-1)} || W x_j^{(k-1)})$$

$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(m_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(m_{ik})}$$

Veličković, Petar, et al. "Graph attention networks." arXiv preprint arXiv:1710.10903 (2017).

Self-attention

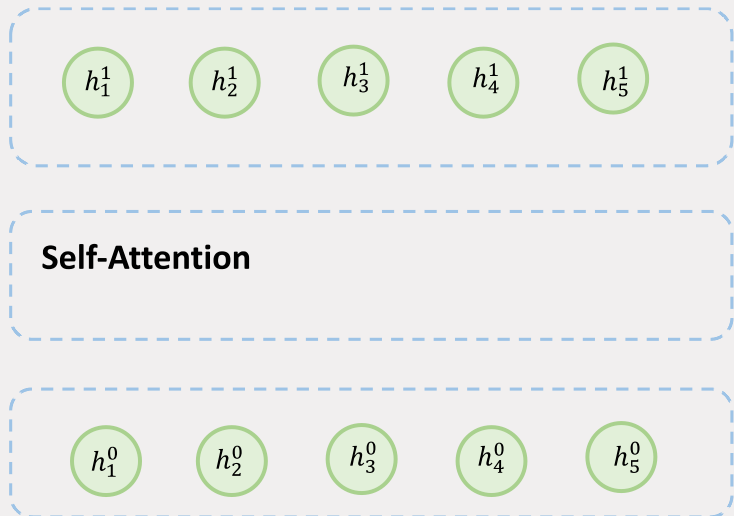


$$e_{ij} = a(W h_i || W h_j)$$

$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}^*(i)} \exp(e_{ik})}$$

$$h'_i = \sigma \left(\sum_{j \in \mathcal{N}^*(i)} \alpha_{ij} W h_j \right)$$

Self-attention

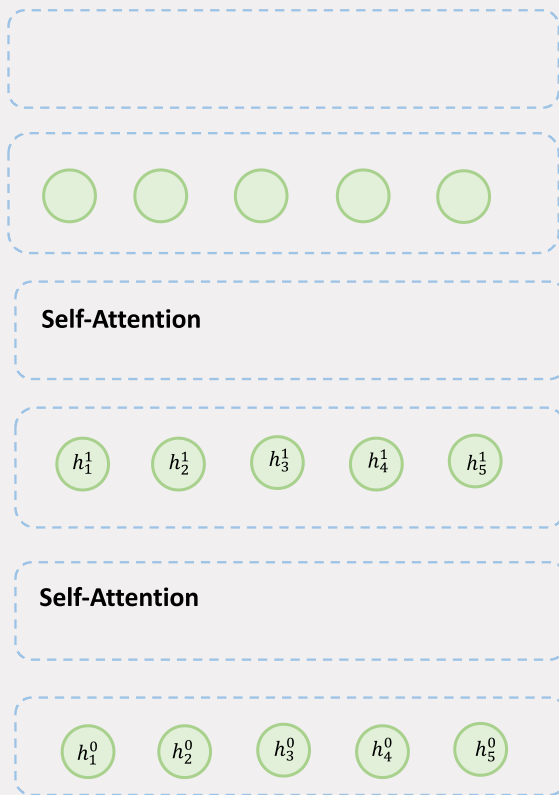


$$e_{ij} = a(W h_i || W h_j)$$

$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}^*(i)} \exp(e_{ik})}$$

$$h'_i = \sigma \left(\sum_{j \in \mathcal{N}^*(i)} \alpha_{ij} W h_j \right)$$

Self-attention



$$e_{ij} = a(W h_i || W h_j)$$

$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}^*(i)} \exp(e_{ik})}$$

$$h'_i = \sigma \left(\sum_{j \in \mathcal{N}^*(i)} \alpha_{ij} W h_j \right)$$

Questions?